

§ 15. 输入输出流 - C语言的文件操作



要求:

- 1、安装UltraEdit软件（**附件已给，版本虽旧，但够用**），学会使用16进制方式查看文件，并掌握ASCII及16进制查看间的切换
 - ★ 已安装VSCode的也可通过相关插件进行16进制方式的查看（**VSCode某种情况下会自动做格式转换或字符集转换，要注意!!!**）
 - ★ 也可以使用其它编辑软件，但**不建议**NotePad++
- 2、完成本文档中所有的测试程序并填写运行结果，从而体会二进制与十进制文件的差异，掌握与文件有关的函数的正确用法
- 3、题目明确指定编译器外，缺省使用VS2022即可（**后缀必须为.c**）
 - ★ 如果要换成其他编译器，可能需要自行修改头文件适配（**不强制要求Linux，但建议试一试**）
 - ★ 部分代码编译时有**warning**，不影响概念理解，**可以忽略**
- 4、直接在本文件上作答，**写出答案/截图（不允许手写、手写拍照截图）**即可；填写答案时，为适应所填内容或贴图，**允许调整**页面的字体大小、颜色、文本框的位置等
 - ★ 贴图要有效部分即可，不需要全部内容
 - ★ 在保证一页一题的前提下，具体页面布局可以自行发挥，简单易读即可
 - ★ **不允许**手写在纸上，再拍照贴图
 - ★ **允许**在各种软件工具上完成（不含手写），再截图贴图
 - ★ 如果某题要求VS+Dev的，则如果两个编译器运行结果一致，贴VS的一张图即可，如果不一致，则两个图都要贴
- 5、转换为pdf后提交
- 6、**11月14日前**网上提交本次作业（在“文档作业”中提交）

特别说明:

- ★ 因为篇幅问题，打开文件后均省略了是否打开成功的判断，这在实际应用中是**不允许**的

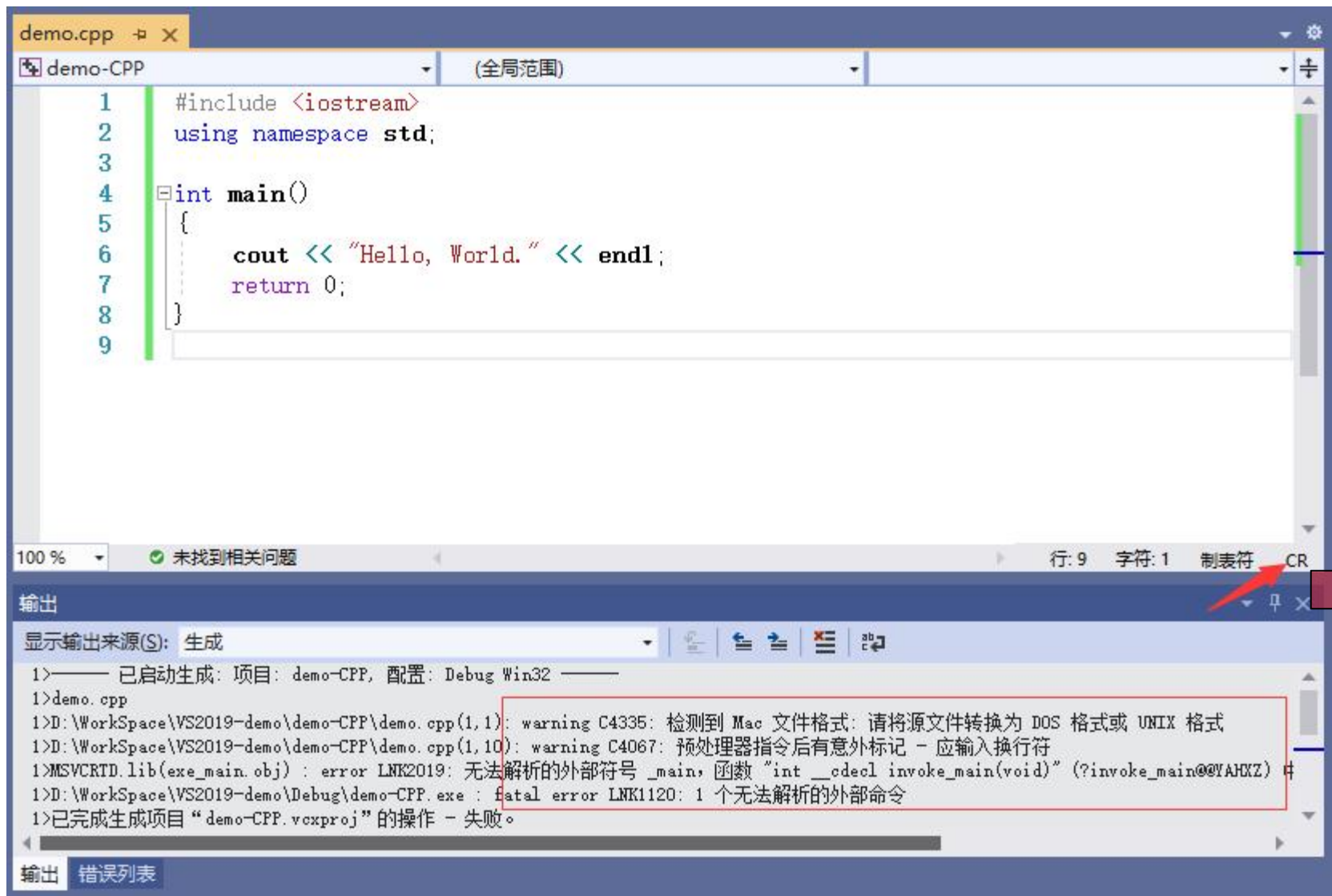


§ 15. 输入输出流 - C语言的文件操作

注意:

附1: 用WPS等其他第三方软件打开PPT, 将代码复制到VS2022中后, 如果出现类似下面的**编译报错**, 则观察源程序编辑窗

的右下角是否为CR, 如果是, 单击CR, 在弹出中选择CRLF, 再次CTRL+F5运行即可

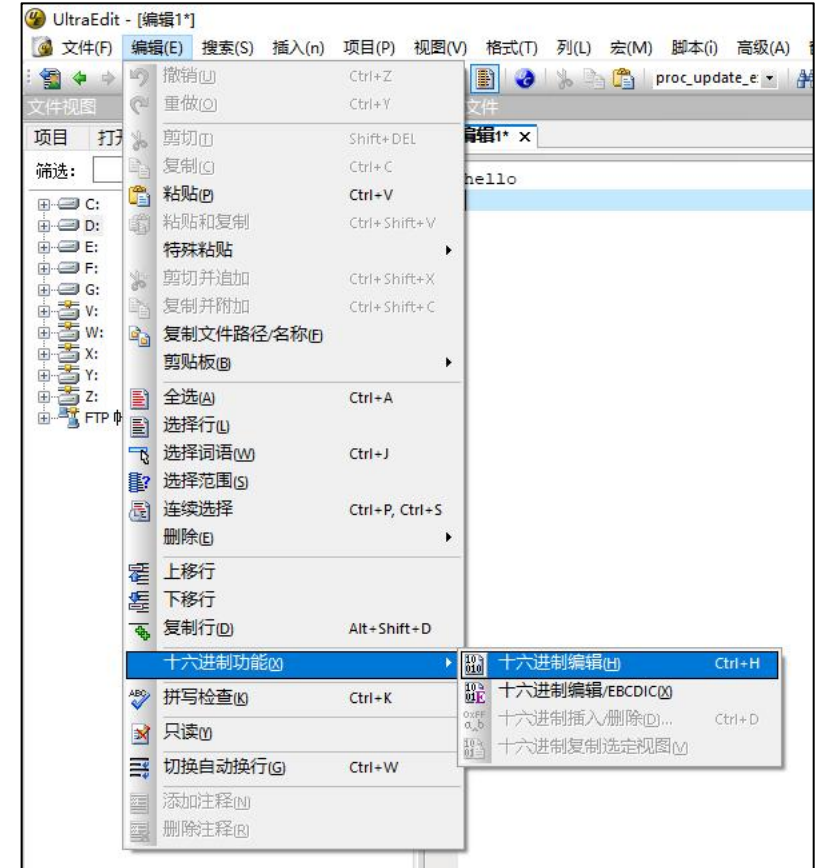
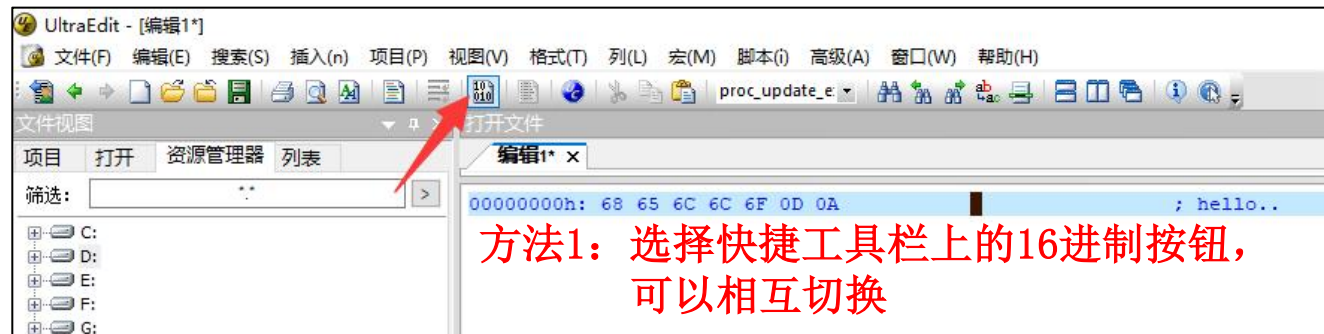
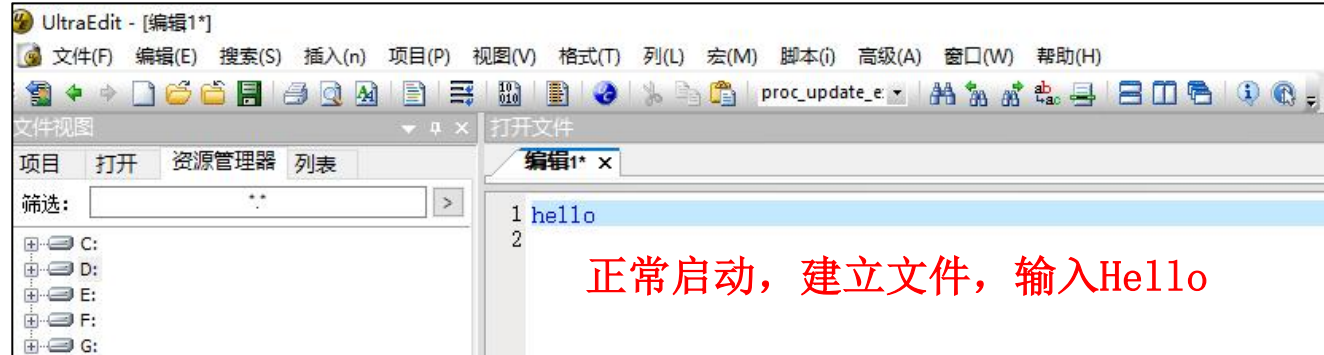




§ 15. 输入输出流 - C语言的文件操作

注意:

附2: 附件给出的UltraEdit查看文件的16进制形式的方法 (三种)



方法2: “编辑” - “十六进制功能” 菜单,
可以相互切换

方法3: Ctrl + H 快捷键可以相互切换



§ 15. 输入输出流 - C语言的文件操作

例1: 十进制方式写

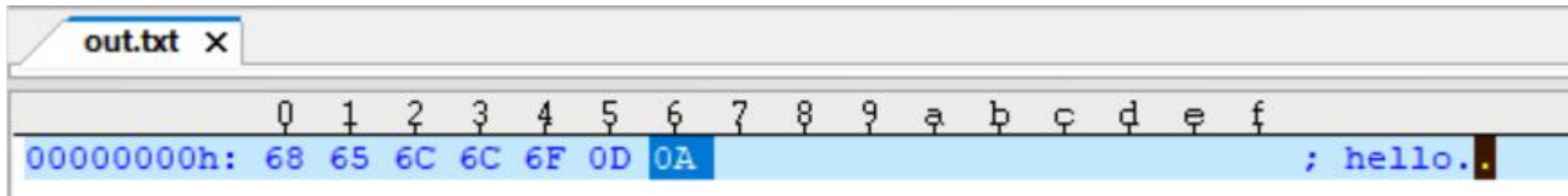
```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>

int main()
{
    FILE *fout;

    fout = fopen("out.txt", "w");
    fprintf(fout, "hello\n");
    fclose(fout);

    return 0;
}
```

Windows下运行, out.txt是__7__字节, 用UltraEdit的16进制方式打开的贴图



本页需填写答案



§ 15. 输入输出流 - C语言的文件操作

例2: 二进制方式写

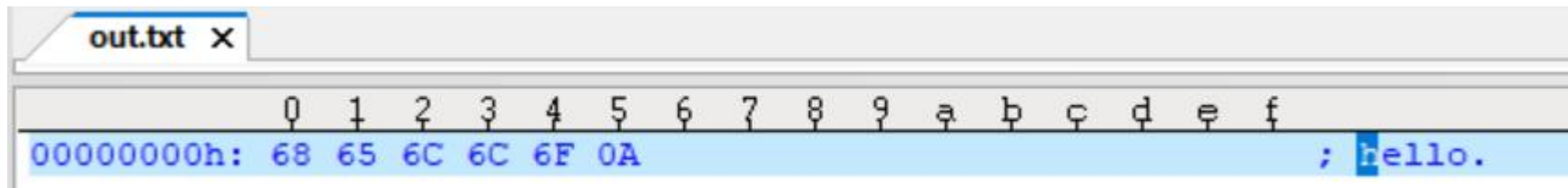
```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>

int main()
{
    FILE *fout;

    fout = fopen("out.txt", "wb");
    fprintf(fout, "hello\n");
    fclose(fout);

    return 0;
}
```

Windows下运行, out.txt是__6__字节, 用UltraEdit的16进制方式打开的贴图



本页需填写答案



§ 15. 输入输出流 - C语言的文件操作

例3: 十进制方式写, 十进制方式读, 0D0A在Windows下的表现

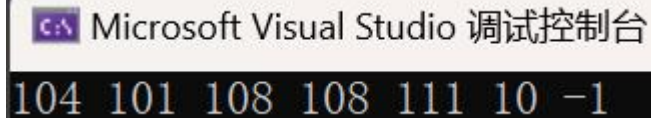
```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>

int main()
{
    FILE *fout, *fin;

    fout = fopen("out.txt", "w");
    fprintf(fout, "hello\n");
    fclose(fout);

    fin = fopen("out.txt", "r");
    while (!feof(fin))
        printf("%d ", fgetc(fin));
    printf("\n");
    fclose(fin);
    return 0;
}
```

Windows下运行, 输出结果是:



Microsoft Visual Studio 调试控制台

104 101 108 108 111 10 -1

说明: 0D 0A在Windows的十进制方式下被当做__1__个字符处理, 值是__\n__。

本页需填写答案



§ 15. 输入输出流 - C语言的文件操作

例4: 十进制方式写，二进制方式读，0D0A在Windows下的表现

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>

int main()
{
    FILE *fout, *fin;

    fout = fopen("out.txt", "w");
    fprintf(fout, "hello\n");
    fclose(fout);

    fin = fopen("out.txt", "rb");
    while (!feof(fin))
        printf("%d ", fgetc(fin));
    printf("\n");
    fclose(fin);
    return 0;
}
```

Windows下运行，输出结果是：



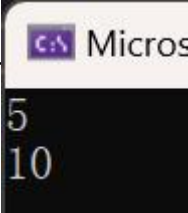
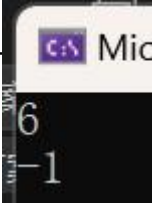
说明：0D 0A在Windows的二进制方式下被当做__2__个字符处理，值是__\r\n__。

本页需填写答案



§ 15. 输入输出流 - C语言的文件操作

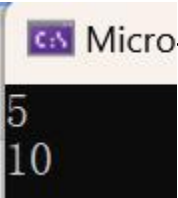
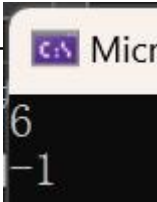
例5: 十进制方式写, 十进制方式读, 不同读方式在Windows下的表现

<pre>#define _CRT_SECURE_NO_WARNINGS #include <stdio.h> #include <string.h> int main() { FILE *fout, *fin; fout = fopen("out.txt", "w"); fprintf(fout, "hello\n"); fclose(fout); char str[80]; fin = fopen("out.txt", "r"); fscanf(fin, "%s", str); printf("%d\n", strlen(str)); printf("%d\n", fgetc(fin)); fclose(fin); return 0; }</pre>	<pre>#define _CRT_SECURE_NO_WARNINGS #include <stdio.h> #include <string.h> int main() { FILE *fout, *fin; fout = fopen("out.txt", "w"); fprintf(fout, "hello\n"); fclose(fout); char str[80]; fin = fopen("out.txt", "r"); fgets(str, sizeof(str), fin); //课上未讲, 自行查阅 printf("%d\n", strlen(str)); printf("%d\n", fgetc(fin)); fclose(fin); return 0; }</pre>
Windows下运行, 输出结果是: 	Windows下运行, 输出结果是: 
说明: fscanf() 读到__\n__就结束了, __\n__还被留在缓冲区中, 因此fgetc() 读到了__\n__。	说明: fgets() 读到__\n__就结束了, __\n__被读掉, 因此fgetc() 读到了__EOF(-1)__。



§ 15. 输入输出流 - C语言的文件操作

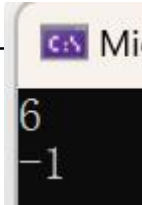
例6: 二进制方式写，十进制方式读，不同读方式在Windows下的表现

<pre>#define _CRT_SECURE_NO_WARNINGS #include <stdio.h> #include <string.h> int main() { FILE *fout, *fin; fout = fopen("out.txt", "wb"); fprintf(fout, "hello\n"); fclose(fout); char str[80]; fin = fopen("out.txt", "r"); fscanf(fin, "%s", str); printf("%d\n", strlen(str)); printf("%d\n", fgetc(fin)); fclose(fin); return 0; }</pre>	<pre>#define _CRT_SECURE_NO_WARNINGS #include <stdio.h> #include <string.h> int main() { FILE *fout, *fin; fout = fopen("out.txt", "wb"); fprintf(fout, "hello\n"); fclose(fout); char str[80]; fin = fopen("out.txt", "r"); fgets(str, sizeof(str), fin); //课上未讲，自行查阅 printf("%d\n", strlen(str)); printf("%d\n", fgetc(fin)); fclose(fin); return 0; }</pre>
Windows下运行，输出结果是： 	Windows下运行，输出结果是： 
说明：fscanf() 读到__\n__就结束了，__\n__还被留在缓冲区中，因此fgetc() 读到了__\n__。	说明：fgets() 读到__\n__就结束了，__\n__被读掉，因此fgetc() 读到了__EOF(-1)__。



§ 15. 输入输出流 - C语言的文件操作

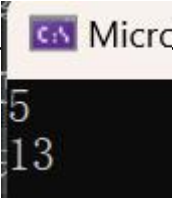
例7：二进制方式写，二进制方式读，不同读方式在Windows下的表现

<pre>#define _CRT_SECURE_NO_WARNINGS #include <stdio.h> #include <string.h> int main() { FILE *fout, *fin; fout = fopen("out.txt", "wb"); fprintf(fout, "hello\n"); fclose(fout); char str[80]; fin = fopen("out.txt", "rb"); fscanf(fin, "%s", str); printf("%d\n", strlen(str)); printf("%d\n", fgetc(fin)); fclose(fin); return 0; }</pre>	<pre>#define _CRT_SECURE_NO_WARNINGS #include <stdio.h> #include <string.h> int main() { FILE *fout, *fin; fout = fopen("out.txt", "wb"); fprintf(fout, "hello\n"); fclose(fout); char str[80]; fin = fopen("out.txt", "rb"); fgets(str, sizeof(str), fin); //课上未讲，自行查阅 printf("%d\n", strlen(str)); printf("%d\n", fgetc(fin)); fclose(fin); return 0; }</pre>
Windows下运行，输出结果是： 	Windows下运行，输出结果是： 
说明：fscanf() 读到__\n__就结束了，__\n__还被留在缓冲区中，因此fgetc() 读到了__\n__。	说明：fgets() 读到__\n__就结束了，__\n__被读掉，因此fgetc() 读到了__EOF(-1)__。



§ 15. 输入输出流 - C语言的文件操作

例8：十进制方式写，二进制方式读，不同读方式在Windows下的表现

<pre>#define _CRT_SECURE_NO_WARNINGS #include <stdio.h> #include <string.h> int main() { FILE *fout, *fin; fout = fopen("out.txt", "w"); fprintf(fout, "hello\n"); fclose(fout); char str[80]; fin = fopen("out.txt", "rb"); fscanf(fin, "%s", str); printf("%d\n", strlen(str)); printf("%d\n", fgetc(fin)); fclose(fin); return 0; }</pre>	<pre>#define _CRT_SECURE_NO_WARNINGS #include <stdio.h> #include <string.h> int main() { FILE *fout, *fin; fout = fopen("out.txt", "w"); fprintf(fout, "hello\n"); fclose(fout); char str[80]; fin = fopen("out.txt", "rb"); fgets(str, sizeof(str), fin); //课上未讲，自行查阅 printf("%d\n", strlen(str)); printf("%d\n", fgetc(fin)); fclose(fin); return 0; }</pre>
Windows下运行，输出结果是： 	Windows下运行，输出结果是： 
说明：fscanf() 读到__\r__就结束了，__\r__还被留在缓冲区中，因此fgetc() 读到了__\r__。	说明：fgets() 读到__\n__就结束了，__\n__被读掉，因此fgetc() 读到了__EOF(-1)__。



§ 15. 输入输出流 - C语言的文件操作

例9: 用十进制方式写入含\0的文件, 观察文件长度

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>

int main()
{
    FILE *fout;

    fout = fopen("out.txt", "w");
    fprintf(fout, "ABC\0\x61\x62\x63");
    fclose(fout);

    return 0;
}
```

Windows下运行, out.txt的大小是__3__字节, 为什么?

向文件中输出"ABC\0\x61\x62\x63"这一字符串时, \0被视为字符串的终止标志。因此, 字符串在遇到\0时即被视为结束, 后面的字符不会被写入。因此, out.txt中只有ABC三个字符。

用UltraEdit的16进制方式显示的截图:



本页需填写答案



§ 15. 输入输出流 - C语言的文件操作

例10: 用十进制方式写入含非图形字符(ASCII码32是空格, 33-126为图形字符), 但不含\0

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>

int main()
{
    FILE *fout;

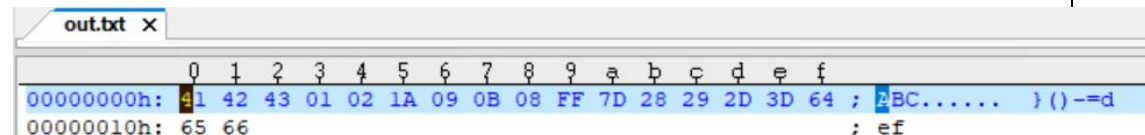
    fout = fopen("out.txt", "w");
    fprintf(fout, "ABC\x1\x2\x1A\t\v\b\xff\175()-=def");
    fclose(fout);

    return 0;
}
```

Windows下运行(VS有warning), out.txt的大小是__18__字节, 为什么?

c语言十进制方式写入字符串, 中间没有尾零不会终止, 字符串一共18个字符。

用UltraEdit的16进制方式显示的截图:



VS的warning是: warning C4819: 该文件包含不能在当前代码页(936)中表示的字符。请将该文件保存为 Unicode 格式以防止数据丢失

VS的哪个字符导致了warning? \xff导致的

本页需填写答案



§ 15. 输入输出流 - C语言的文件操作

例11: 用十进制方式写入含\x1A(十进制26=CTRL+Z)的文件, 并用十进制/二进制方式读取

<pre>#define _CRT_SECURE_NO_WARNINGS #include <stdio.h> int main() { FILE *fout, *fin; fout = fopen("out.txt", "w"); fprintf(fout, "ABC\x1\x2\x1A\t\v\b\175()--def"); fclose(fout); fin = fopen("out.txt", "r"); int c=0; while(!feof(fin)) { fgetc(fin); c++; } printf("c=%d\n", c); fclose(fin); return 0; }</pre>	<pre>#define _CRT_SECURE_NO_WARNINGS #include <stdio.h> int main() { FILE *fout, *fin; fout = fopen("out.txt", "w"); fprintf(fout, "ABC\x1\x2\x1A\t\v\b\175()--def"); fclose(fout); fin = fopen("out.txt", "rb"); int c=0; while(!feof(fin)) { fgetc(fin); c++; } printf("c=%d\n", c); fclose(fin); return 0; }</pre>
Windows下运行, 文件大小: <u>17</u> 输出的c是: <u>6</u>	Windows下运行, 文件大小: <u>17</u> 输出的c是: <u>18</u>
为什么? 0x1A被认为是EOF文件结束标志, 所以在fgetc读到\x1A时, in.eof为真, 循环结束, c为6	c的大小比文件大小大__1__, 原因是: 程序中第一个字符为EOF时, 还没读取到EOF, in.eof为假, 循环会多做一遍, 直到fgetc读到EOF才会终止

本页需填写答案



§ 15. 输入输出流 - C语言的文件操作

例11: (改) 只允许修改while循环, 使“c=xx”的输出 (左侧: \x1A前字符数, 右侧: 与文件大小一致)

<pre>#define _CRT_SECURE_NO_WARNINGS #include <stdio.h> int main() { FILE *fout, *fin; fout = fopen("out.txt", "w"); fprintf(fout, "ABC\x1\x2\x1A\t\v\b\175()--def"); fclose(fout); fin = fopen("out.txt", "r"); int c=0; while(!feof(fin)) { fgetc(fin); c++; } printf("c=%d\n", c); fclose(fin); return 0; }</pre>		<pre>#define _CRT_SECURE_NO_WARNINGS #include <stdio.h> int main() { FILE *fout, *fin; fout = fopen("out.txt", "w"); fprintf(fout, "ABC\x1\x2\x1A\t\v\b\175()--def"); fclose(fout); fin = fopen("out.txt", "rb"); int c=0; while(!feof(fin)) { fgetc(fin); c++; } printf("c=%d\n", c); fclose(fin); return 0; }</pre>	
Windows下运行, 文件大小: 17 输出的c是: 5		Windows下运行, 文件大小: 17 输出的c是: 17	

本页需填写答案



§ 15. 输入输出流 - C语言的文件操作

例12: 用十进制方式写入含\xFF(十进制255/-1, EOF的定义是-1)的文件, 并用十/二进制读取

<pre>#define _CRT_SECURE_NO_WARNINGS #include <stdio.h> int main() { FILE *fout, *fin; fout = fopen("out.txt", "w"); fprintf(fout, "ABC\x1\x2\xff\t\v\b\175()--def"); fclose(fout); fin = fopen("out.txt", "r"); int c=0; while(!feof(fin)) { fgetc(fin); c++; } printf("c=%d\n", c); fclose(fin); return 0; }</pre>	<pre>#define _CRT_SECURE_NO_WARNINGS #include <stdio.h> int main() { FILE *fout, *fin; fout = fopen("out.txt", "w"); fprintf(fout, "ABC\x1\x2\xff\t\v\b\175()--def"); fclose(fout); fin = fopen("out.txt", "rb"); int c=0; while(!feof(fin)) { fgetc(fin); c++; } printf("c=%d\n", c); fclose(fin); return 0; }</pre>
Windows下运行, 文件大小: 17 输出的c是: 18	Windows下运行, 文件大小: 17 输出的c是: 18

综合例11~例12, 结论: 当文件中含字符__0x1A__ (0x1A/0xFF) 时, 不能用十进制方式读取, 而当文件中含字符__0xFF__ (0x1A/0xFF) 时, 是可以用二/十进制方式正确读取的

本页需填写答案



§ 15. 输入输出流 - C语言的文件操作

例12: (改) 只允许修改while循环, 使“c=xx”的输出与文件大小一致

<pre>#define _CRT_SECURE_NO_WARNINGS #include <stdio.h> int main() { FILE *fout, *fin; fout = fopen("out.txt", "w"); fprintf(fout, "ABC\x1\x2\xFF\t\v\b\175()--def"); fclose(fout); fin = fopen("out.txt", "r"); int c=0; while(!feof(fin)) { fgetc(fin); c++; } printf("c=%d\n", c); fclose(fin); return 0; }</pre>		<pre>#define _CRT_SECURE_NO_WARNINGS #include <stdio.h> int main() { FILE *fout, *fin; fout = fopen("out.txt", "w"); fprintf(fout, "ABC\x1\x2\xFF\t\v\b\175()--def"); fclose(fout); fin = fopen("out.txt", "rb"); int c=0; while(!feof(fin)) { fgetc(fin); c++; } printf("c=%d\n", c); fclose(fin); return 0; }</pre>	
Windows下运行, 文件大小: 17 输出的c是: 17		Windows下运行, 文件大小: 17 输出的c是: 17	

本页需填写答案



§ 15. 输入输出流 - C语言的文件操作

例13: 比较格式化读和read()读的区别, 并观察ftell在不同读入方式时值的差别

<pre>#define _CRT_SECURE_NO_WARNINGS #include <stdio.h> int main() { FILE *fout, *fin; fout = fopen("out.txt", "w"); fprintf(fout, "ABCDEFGHJKLMNOPQRSTUVWXYZ\n"); fclose(fout); fin = fopen("out.txt", "rb"); char name[30]; fscanf(fin, "%s", name); printf("%s*\n", name); printf("%d\n", name[26]); printf("%d\n", ftell(fin)); fclose(fin); return 0; }</pre>	<pre>#define _CRT_SECURE_NO_WARNINGS #include <stdio.h> int main() { FILE *fout, *fin; fout = fopen("out.txt", "w"); fprintf(fout, "ABCDEFGHJKLMNOPQRSTUVWXYZ\n"); fclose(fout); fin = fopen("out.txt", "rb"); char name[30]; fread(name, sizeof(name), 1, fin); printf("%s*\n", name); printf("%d\n", name[26]); printf("%d\n", ftell(fin)); fclose(fin); return 0; }</pre>
Windows下运行, 文件大小: 28 输出的name是: ABCDEFGHJKLMNOPQRSTUVWXYZ name[26]的值是: 0 ftell的值是: 26 说明: fscanf方式读入字符串时, 和scanf方式相同, 都是读到\n停止, 并在数组最后加入一个\0。	Windows下运行, 文件大小: 28 输出的name是: ABCDEFGHJKLMNOPQRSTUVWXYZ+乱码 name[26]的值是: 13 ftell的值是: 28 说明: fread()读入时, 是读到\n停止, 不在数组最后加入一个\0。

不要截图, 手填, 确定乱码的地方可以填“乱码”

本页需填写答案



§ 15. 输入输出流 - C语言的文件操作

例14: 比较read() 读超/不超过文件长度时的区别, 并观察ftell()/feof() 的返回值

<pre>#define _CRT_SECURE_NO_WARNINGS #include <stdio.h> int main() { FILE *fout, *fin; fout = fopen("out.txt", "w"); fprintf(fout, "ABCDEFGHJKLMNOPQRSTUVWXYZ");//无\n fclose(fout); fin = fopen("out.txt", "rb"); char name[30] = "00000000000000000000000000000000"; fread(name, 20, 1, fin); printf("%s*\n", name); printf("%d\n", name[20]); printf("%d\n", ftell(fin)); printf("%d\n", feof(fin)); fclose(fin); return 0; }</pre>	<pre>#define _CRT_SECURE_NO_WARNINGS #include <stdio.h> int main() { FILE *fout, *fin; fout = fopen("out.txt", "w"); fprintf(fout, "ABCDEFGHJKLMNOPQRSTUVWXYZ");//无\n fclose(fout); fin = fopen("out.txt", "rb"); char name[30] = "00000000000000000000000000000000"; fread(name, 200, 1, fin); printf("%s*\n", name); printf("%d\n", ftell(fin)); printf("%d\n", feof(fin)); fclose(fin); return 0; }</pre>
Windows下运行, 文件大小: 26 输出的name是: ABCDEFGHJKLMNOPQRST0000000000 name[20]的值是: 48 ftell的值是: 20 feof的值是: 0	Windows下运行, 文件大小: 26 输出的name是: ABCDEFGHJKLMNOPQRSTUVWXYZ000 ftell的值是: 26 feof的值是: 1

本页需填写答案



§ 15. 输入输出流 - C语言的文件操作

例15: 使用fseek()移动文件指针, 观察ftell()不同情况下的返回值

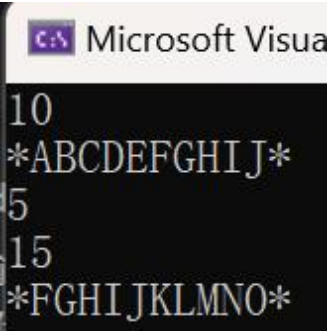
本页需填写答案

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>

int main()
{
    FILE *fout, *fin;
    fout = fopen("out.txt", "w");
    fprintf(fout, "ABCDEFGHIJKLMNOPQRSTUVWXYZ"); //无换行符
    fclose(fout);

    fin = fopen("out.txt", "rb");
    char name[80];
    fread(name, 10, 1, fin);
    printf("%d\n", ftell(fin));
    name[10] = '\0';
    printf("%s*\n", name);
    fseek(fin, -5, SEEK_CUR);
    printf("%d\n", ftell(fin));
    fread(name, 10, 1, fin);
    printf("%d\n", ftell(fin));
    name[10] = '\0';
    printf("%s*\n", name);
    fclose(fin);

    return 0;
}
```



Windows下运行, 输出依次是: (可截图, 需对结果做分析)
从文件中读取10个字符ABCDEFGH IJ到数组 name 中。此时, 文件指针移动了10个字节, 现在位于第10个字节处, 输出为10。给name第十个字符赋值为\0, 输出字符串打印前10个字符, 即为ABCDEFGH IJ*。将文件指针从当前位置向回移动5个字节, 移动到第5个字节处, 输出为5。再次读取10个字符FGHI JKLMNO到 name 数组中, 文件指针随之移动了10个字节, 输出15。给name第十个字符赋值为\0, 输出字符串打印前10个字符, 即为FGHI JKLMNO。

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>

int main()
{
    FILE *fout, *fin;
    fout = fopen("out.txt", "w");
    fprintf(fout, "ABCDEFGHIJKLMNOPQRSTUVWXYZ"); //无换行符
    fclose(fout);

    fin = fopen("out.txt", "rb");
    char name[80];
    fread(name, 30, 1, fin);
    printf("%d\n", ftell(fin));
    name[30] = '\0';
    printf("%s*\n", name);
    fseek(fin, 5, SEEK_SET);
    printf("%d\n", ftell(fin));
    fread(name, 30, 1, fin);
    printf("%d\n", ftell(fin));
    name[30] = '\0';
    printf("%s*\n", name);
    fclose(fin);

    return 0;
}
```



Windows下运行, 输出依次是: (可截图, 需对结果做分析)
从文件中读取30个字符到数组 name 中。由于读取文件中只有26个字符, 文件指针移动了26个字节, 现在位于第26个字节处, 输出为26。给name第30个字符赋值为\0, 输出字符串打印前30个字符, 但name中第26-29个字符未赋值, 所以会输出乱码, 最终输出ABCDEFGH IJKLMNOPQRSTU VWXYZ+乱码。将文件指针从起始位置向后移动5个字节, 移动到第5个字节处, 输出为5。再次读取30个字符, 文件指针为5, 只能从F读到Z, 覆盖name的前21个字符, 其余与上相同, 所以输出FGHI JKLMNOPQRSTU VWXYZ+乱码。



§ 15. 输入输出流 - C语言的文件操作

例16: fread的返回值理解

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>

int main()
{
    FILE *fp;
    char buf[80];

    fp = fopen("in.txt", "r");
    int ret = fread(buf, 26, 1, fp);
    printf("ret=%d\n", ret);
    fclose(fp);

    return 0;
}
```

准备工作: 在当前目录下建in.txt文件,
写入A..Z共26个字母, 不要加回车
确定文件大小为26字节!!!

fread的第2/3参数:

原26, 1, ret=1
换1, 26, ret=23
换13, 2, ret=2
换2, 13, ret=13
换80, 1, ret=0
换1, 80, ret=26
换15, 2, ret=1
换2, 15, ret=13

本页需填写答案



§ 15. 输入输出流 - C语言的文件操作

例17: fread用于二进制/十进制方式打开的文件时的返回值理解

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>

int main()
{
    FILE *fp;
    char buf[80];

    fp = fopen("in.txt", "r");
    int ret = fread(buf, 1, 80, fp);
    printf("ret=%d\n", ret);
    fclose(fp);

    return 0;
}
```

准备工作: 当前目录下建in.txt文件, 写多行

例: abc
123
xyz

注: 1、考虑到字符集问题, 不要中文
2、文件总大小不超过50字节
3、最后一行加不加回车均可

文件编辑完成后, Windows右键菜单查看文件属性, 能看到大小是__13__字节。

运行左侧程序, 打印的ret=__11__

将"r"改为"rb", 再次运行, 打印的ret=__13__

两次运行结果不一样的原因是: 十进制方式读取时, 将换行符的0D 0A看作是一个字节读入; 二进制方式则看成两个字符。文件中共有两个换行符, 所以十进制的ret比二进制的少了2。



§ 15. 输入输出流 - C语言的文件操作

例18: fwrite返回值理解

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>

int main()
{
    FILE *fp;
    char buf[80]="abcdefghijklmnopqrstuvwxyz";

    fp = fopen("out.txt", "w");
    int ret = fwrite(buf, 26, 1, fp);
    printf("ret=%d\n", ret);
    fclose(fp);

    return 0;
}
```

fwrite的第2/3参数:

原26, 1, ret=__1__, 文件大小__26__
换1, 26, ret=__26__, 文件大小__26__
换13, 2, ret=__2__, 文件大小__26__
换2, 13, ret=__13__, 文件大小__26__
换80, 1, ret=__1__, 文件大小__80__
换1, 80, ret=__80__, 文件大小__80__
换15, 2, ret=__2__, 文件大小__30__
换2, 15, ret=__15__, 文件大小__30__



§ 15. 输入输出流 - C语言的文件操作

例19: fwrite用于二进制/十进制方式打开的文件时的返回值理解

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <string.h>

int main()
{
    FILE *fp;
    char buf[80]="abc\n123\nxyz\n";

    fp = fopen("out.txt", "w");
    int ret = fwrite(buf, 1, strlen(buf), fp);
    printf("ret=%d\n", ret);
    fclose(fp);

    return 0;
}
```

运行左侧程序，打印的
ret=__12__，
Windows右键菜单查看文件属性，大小是__15__字节。

将"w"改为"wb"，再次运行，打印的
ret=__12__，Windows右键菜单查看
文件属性，大小是__12__字节。

两次运行打印的ret一样，但文件属性中看到的文件大小不一样的原因是：十进制方式写入时，将换行符作为两个字节0D 0A写入；二进制方式则作为一个字符0A写入。一共写入了三个换行符，所以十进制的文件大小比二进制的大了3。但一共都是写入了12个字符，所以ret返回值都是12。